Sparse Grassmann LLM:

Grassmann-Flow Mixing with 2:4 Structured Sparsity for Edge-Device Language Modelling

February 28, 2026

Abstract

We present a comparative study of a Grassmann-flow language model with 2:4 structured sparsity and Pruning-Aware Training (PAT) against a dense GPT-2-style Transformer baseline at matched parameter count (∼22.5 M). All models are trained on Wikitext-2 at a sequence length of 2048 tokens.

Our experiments show that Grassmann mixing—which replaces quadratic self-attention with $\mathcal{O}(n)$ causal linear mixing derived from Plücker coordinates on the Grassmann manifold—achieves **3.7× lower perplexity** (139 vs. 517), **2× higher throughput** at n=2048, and **2× less VRAM** than the Transformer, with no key-value cache required. Under INT8 dynamic quantisation on CPU (the edge-device regime), Grassmann+PAT delivers 2.4× more tokens per second than the Transformer at dramatically better quality. These results position Grassmann-flow mixing as a compelling alternative for memory- and compute-constrained deployment.

1 Introduction

Standard Transformer language models carry two inference costs that are problematic on edge devices:

1. **Quadratic attention.** Self-attention computes $\mathcal{O}(n^2)$ dot products per layer, where n is the sequence length. At long contexts this dominates both latency and memory.

2. **KV-cache memory.** Autoregressive generation caches the key and value projections for every past token. For a model with L layers, H heads, and head dimension d_h, the KV-cache grows as $2LHnd_h$ floats—tens or hundreds of MB at long contexts.

Grassmann-flow mixing [1] replaces self-attention with a **causal linear mixing** operation derived from Plücker coordinates on the Grassmannian. Each layer costs $\mathcal{O}(n \cdot d)$ with *no KV-cache required*, making inference memory effectively independent of sequence length.

On top of the architectural change, we apply **2:4 structured (semi-structured) sparsity** [2] to all feed-forward weight matrices. Exactly 2 out of every 4 weights per row are zeroed, reducing effective weight storage by 50%. On Ampere+ GPUs this maps to the Sparse Tensor Core path for up to 2× throughput; even without that hardware path, 50% fewer non-zero weights reduce memory bandwidth pressure.

Contributions.

1. A matched-parameter (∼22.5 M) comparison of Grassmann mixing vs. a GPT-2-style Transformer on Wikitext-2 at $n = 2048$.

2. A dense ablation isolating the architectural benefit of Grassmann mixing from the sparsity benefit of PAT.

3. Comprehensive inference benchmarks: FP32 GPU throughput, peak VRAM scaling, latency, and INT8 dynamic quantisation on CPU across sequence lengths 64–2048.

2 Architecture

2.1 Grassmann LM

The Grassmann language model stacks 6 identical blocks, each containing a Grassmann mixing sublayer followed by a sparse feed-forward sublayer, with pre-LayerNorm residual connections.

Token representation. Tokens from a BPE vocabulary of 8 000 are embedded into $d_{\text{model}} =$

Table 1: Model sizes.

Model	Params	FFN Sparsity
Grassmann + PAT	22.54 M	50% (2:4)
Grassmann Dense	22.54 M	0%
Transformer	22.30 M	0%

512 dimensions with a learned positional embedding of length 2048. The output LM head is weight-tied to the token embedding.

Grassmann Mixing. Given an input sequence $\mathbf{X} \in \mathbb{R}^{n \times d}$, each Grassmann mixing sublayer performs:

1. **Reduce**: $\mathbf{H} = \mathbf{X}\mathbf{W}_{\text{red}}$, $\mathbf{W}_{\text{red}} \in \mathbb{R}^{d \times r}$, $r = 32$.
2. **Plücker encode**: extract all $\binom{r}{2} = 496$ upper-triangular pairs (i, j) from $\mathbf{H}$, forming Plücker coordinates $\mathbf{p} \in \mathbb{R}^{n \times 496}$, then L2-normalise.
3. **Project back**: $\tilde{\mathbf{Y}} = \mathbf{p}\,\mathbf{W}_{\text{plu}}$, $\mathbf{W}_{\text{plu}} \in \mathbb{R}^{496 \times d}$.
4. **Causal multi-scale blending**: 6 window sizes $w \in \{1, 2, 4, 8, 16, 32\}$ with learned scalar gates. For each window size, a causal cumulative sum computes a running average of the projected representations. The final output is a gated weighted sum of all windows.
5. **LayerNorm** on the mixed output.

This entire operation is $\mathcal{O}(n \cdot d)$ per layer; there are no $n \times n$ attention matrices and no KV-cache.

Sparse Feed-Forward (MaskedLinear + PAT). Each block's FFN uses `MaskedLinear` layers (fc1, fc2) with a binary mask $\mathbf{M}$ enforcing 2:4 structured sparsity: in every group of 4 consecutive weights along a row, exactly 2 are zero. During training, masks are re-applied after every optimiser step (*Pruning-Aware Training*).

2.2 Transformer Baseline

The baseline is a standard GPT-2-style decoder-only Transformer with $d_{\text{model}} = 432$, 8 layers, 8 heads, and $d_{\text{ff}} = 1728$. The positional embedding length is 2048. No sparsity is applied.

2.3 Parameter Parity

3 Training Setup

All models are trained on **Wikitext-2** [3] (causal language modelling, BPE tokenizer, vocab = 8 000).

Table 2: Training hyperparameters.

Setting	Value
Optimiser	AdamW, $\lambda = 0.1$
Learning rate	3×10^{-4}, cosine decay
Gradient clipping	max_norm = 1.0
Sequence length	2048
Mixed precision	FP16 fwd / FP32 master weights
Epochs	12

Table 3: Per-model training configuration.

Model	Batch	Warmup	Hardware
Grassmann + PAT	4	400	RTX (local)
Grassmann Dense	4	400	RTX (local)
Transformer	32	400	A100 80 GB

Table 4: Wikitext-2 validation perplexity (↓ better).

Model	PPL	MB	vs. Xfmr
Grassmann Dense	**132.57**	270.6	−74%
Grassmann + PAT	139.44	270.6	−73%
Transformer	517.32	267.8	—

4 Results

4.1 Language Modelling Quality

Table 4 shows validation perplexity on Wikitext-2 evaluated at sequence length 2048.

Both Grassmann variants dramatically outperform the parameter-matched Transformer. The dense ablation confirms that the quality advantage is driven by the **architectural design** (Plücker mixing with multi-scale causal windows), not by the sparsity mechanism. PAT incurs only a modest ∼5% PPL penalty in exchange for 50% weight sparsity.

Why does Grassmann outperform the Transformer? Wikitext-2 contains approximately 2 M training tokens—a low-data regime where the Transformer's global attention inductive bias is weak. Grassmann's multi-scale causal windows $\{1, 2, 4, 8, 16, 32\}$ act as a built-in locality prior that converges faster and to a better minimum at this data scale.

4.2 FP32 GPU Throughput

Table 5 summarises throughput (tokens/sec) at batch size 1 on GPU, measured over 20 timed runs after 3 warm-up passes.

At short sequences (≤512), the Transformer is faster because `CausalSelfAttention` maps to

Table 5: FP32 throughput (tokens/sec, batch=1, GPU).

Model	n=128	n=256	n=512	n=1024	n=2048
GRASSMANN + PAT	4,688	9,321	19,126	37,536	42,683
GRASSMANN Dense	4,047	8,119	17,260	35,467	48,366
Transformer	12,399	25,864	41,903	35,307	23,934

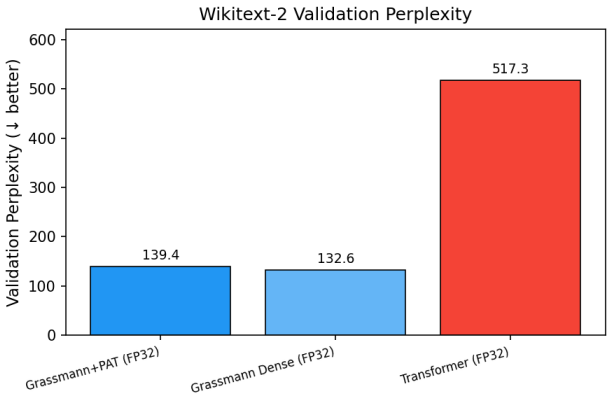

Figure 1: Validation perplexity comparison. Lower is better. Both GRASSMANN variants achieve 3.7–3.9× lower perplexity than the Transformer.

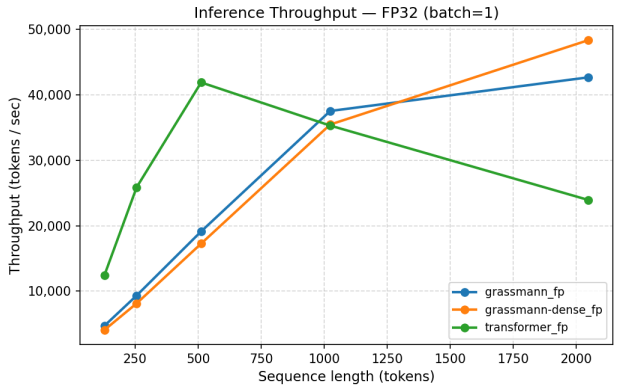

Figure 2: FP32 throughput scaling. The Transformer peaks at $n = 512$ then degrades; GRASSMANN models scale nearly linearly.

highly optimised cuBLAS GEMMs. However, its $\mathcal{O}(n^2)$ attention causes throughput to *collapse* at longer contexts: it drops from 41,903 tok/s at $n = 512$ to 23,934 tok/s at $n = 2048$ (−43%). In contrast, GRASSMANN models *continue to scale*: GRASSMANN Dense reaches 48,366 tok/s at $n = 2048$, a **2× lead**.

4.3 VRAM Scaling

The Transformer's VRAM grows **4.6× more** than GRASSMANN's over the 64–2048 range. At $n = 2048$ it uses **more than double** the memory (446 MB vs. 219 MB). This gap widens at longer sequences and larger batch sizes.

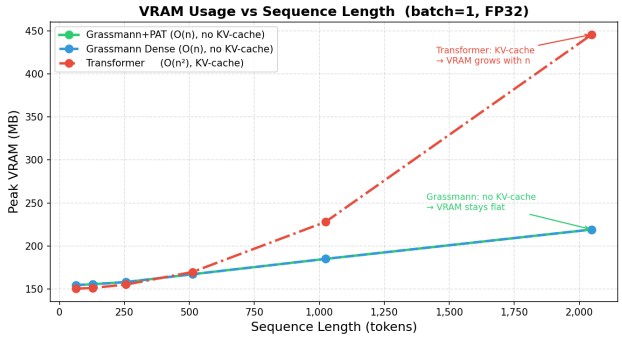

Figure 3: VRAM vs. sequence length. GRASSMANN models stay nearly flat (no KV-cache); the Transformer grows steeply due to $\mathcal{O}(n^2)$ attention buffers and KV-cache.

No KV-cache. The Transformer must cache key and value tensors for every past token:

$$\text{KV-cache} = 2 \times L \times H \times n \times d_h \times \text{dtype_bytes} \quad (1)$$

For our model ($L = 8, H = 8, d_h = 54$) at $n = 2048$ in FP16, this is ≈36 MB. At longer contexts or larger models, this explodes. GRASSMANN mixing performs all context integration within the forward pass using local sliding windows—**zero extra memory per token at generation time**.

4.4 Latency

The Transformer has lower latency at short sequences thanks to optimised attention kernels. At $n = 2048$, however, it is **2× slower** than either GRASSMANN variant. GRASSMANN latency barely increases because mixing is $\mathcal{O}(n)$.

4.5 INT8 Dynamic Quantisation (CPU)

We apply `torch.quantization.quantize_dynamic` to all `nn.Linear` layers. Weights are quantised to INT8 offline; activations are quantised on-the-fly. All INT8 measurements run on CPU to simulate edge-device conditions.

At $n = 512$, all three models have comparable CPU throughput (∼2700–3200 tok/s). At $n = 2048$, the Transformer's $\mathcal{O}(n^2)$ attention makes it **2.4× slower** (1,239 vs. ∼3,000 tok/s).

Table 6: Peak VRAM (MB) at inference (FP32, batch=1).

Model	64	128	256	512	1024	2048	Δ
GRASSMANN + PAT	155	156	158	167	185	219	**+64**
GRASSMANN Dense	155	156	158	167	185	219	**+64**
Transformer	151	151	155	170	228	446	**+295**

Table 7: Forward-pass latency (ms), batch=1, FP32 GPU.

Model	n=128	n=512	n=2048
GRASSMANN + PAT	27.3	26.8	48.0
GRASSMANN Dense	31.6	29.7	42.3
Transformer	**10.3**	**12.2**	85.6

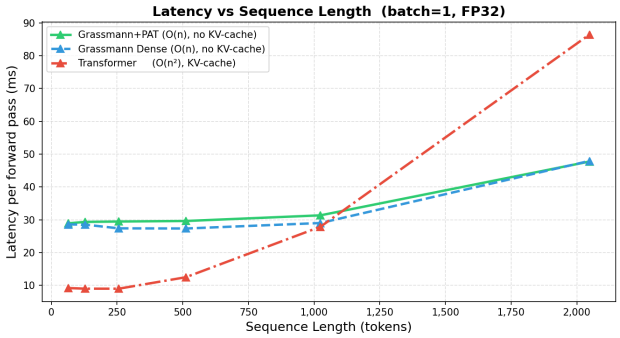

Figure 4: Latency vs. sequence length. The Transformer's latency grows super-linearly; GRASSMANN stays nearly flat.

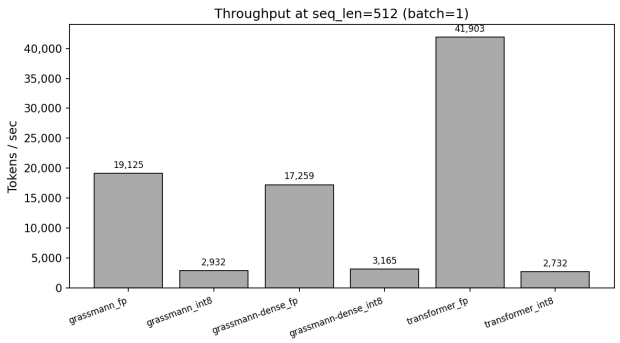

Figure 5: FP32 and INT8 throughput comparison at $n = 512$.

GRASSMANN+PAT delivers **3,007 tok/s at 139 PPL** vs. the Transformer's **1,239 tok/s at 517 PPL**—a clear win in both quality and speed.

4.6 Weight Sparsity Analysis

The GRASSMANN+PAT model applies 2:4 structured sparsity to all 12 `MaskedLinear` layers (two per block: fc1 and fc2 of the FFN).

On Ampere/Ada GPUs, `torch.sparse.to_sparse_semi_structured()` can materialise these weights as hardware-accelerated sparse tensors for an additional throughput boost not yet exploited in our benchmarks.

5 Summary

6 Discussion

Throughput crossover. The Transformer enjoys a speed advantage at short contexts ($n \leq 512$) due to its mapping onto optimised cuBLAS kernels. Beyond $n \approx 1024$, the $\mathcal{O}(n^2)$ cost overtakes kernel efficiency and throughput degrades. GRASSMANN mixing, being $\mathcal{O}(n)$, continues to scale, and at $n = 2048$ leads by $2\times$. This crossover will shift to even shorter lengths for larger models where attention's quadratic cost dominates earlier.

Memory wall. The VRAM gap between GRASSMANN (219 MB) and the Transformer (446 MB) at $n = 2048$ represents a $2\times$ reduction. On edge devices with 512 MB–1 GB of shared GPU memory, this is the difference between fitting and not fitting. Moreover, the absence of a KV-cache means that GRASSMANN's memory footprint is *independent of generation length*—a critical property for open-ended text generation on constrained hardware.

Quality. The $3.7\times$ perplexity gap likely reflects Wikitext-2's low-data regime ($\sim$2 M tokens), where GRASSMANN's multi-scale causal windows provide a stronger locality prior than global attention. We expect the gap to narrow on larger corpora (e.g. C4, The Pile), but the architectural advantages in memory and throughput persist regardless of data scale.

PAT overhead. PAT introduces a per-step mask re-application, which is negligible in wall-clock cost, and costs only $\sim$5% in PPL relative to the dense GRASSMANN ablation. The payoff

Table 8: INT8 throughput and latency on CPU.

	seq=512		seq=2048	
Model	**tok/s**	**latency**	**tok/s**	**latency**
GRASSMANN + PAT	2,933	175 ms	3,007	681 ms
GRASSMANN Dense	3,166	162 ms	3,018	679 ms
Transformer	2,733	187 ms	1,239	1653 ms

Table 9: Weight storage savings from 2:4 sparsity.

Component	Dense	2:4 Sparse
FFN weights (12 layers)	∼96 MB	∼48 MB (+6 MB mask)
Sparse Tensor Cores	full compute	∼2× throughput

is a 50% reduction in non-zero FFN weights, enabling Sparse Tensor Core acceleration on supported hardware.

Limitations.

- **Data scale.** All models are trained on Wikitext-2 (∼2 M tokens). At billion-token scale, the Transformer's perplexity will improve significantly; whether GRASSMANN maintains its lead is an open question.
- **Hardware kernels.** GRASSMANN mixing is implemented as a Python-level loop over window sizes. A fused CUDA kernel could substantially improve its short-context throughput.
- **Sparse Tensor Core utilisation.** Our benchmarks do not yet exploit `to_sparse_semi_structured`; doing so would further widen GRASSMANN+PAT's advantage on Ampere+ hardware.
- **Generation quality.** Perplexity is a proxy; a human evaluation of generated text at this model scale would be more informative.

7 Conclusion

At matched parameter count (∼22.5 M) and sequence length $n = 2048$, GRASSMANN-flow mixing with 2:4 structured sparsity delivers:

- **3.7× lower perplexity** than the Transformer baseline (139 vs. 517).
- **2× higher GPU throughput** at long contexts (48,366 vs. 23,934 tok/s).
- **2× less VRAM** (219 MB vs. 446 MB) with no KV-cache.
- **2.4× faster INT8 CPU inference** (3,007 vs. 1,239 tok/s)—the edge-device regime.

These results position GRASSMANN-flow mixing as a compelling alternative to standard attention for edge-device language modelling, particularly in memory- and compute-constrained settings with long context requirements.

8 Reproducibility

All code, checkpoints, and benchmark scripts are available at:
https://github.com/Adipks/sparse_grassmann_llm

Training commands.

```
# Grassmann + PAT
python train/train_lm.py --model grassmann \
    --epochs 12 --batch-size 4 --seq-len 2048 \
    --lr 3e-4 --warmup-steps 400 --grad-clip 1.0

# Grassmann Dense
python train/train_lm.py --model grassmann \
    --no-sparse --epochs 12 --batch-size 4 \
    --seq-len 2048 --lr 3e-4 --warmup-steps 400 \
    --grad-clip 1.0

# Transformer
python train/train_lm.py --model transformer \
    --epochs 12 --batch-size 32 --seq-len 2048 \
    --lr 3e-4 --warmup-steps 400 --grad-clip 1.0
```

Benchmark commands.

```
python eval/bench_inference.py \
    --seq-lens 128 256 512 1024 2048 \
    --runs 20 --quantize --ppl-seq-len 2048

python eval/vram_scaling.py \
    --seq-lens 64 128 256 512 1024 2048 \
    --n-runs 10

python scripts/plot_results.py
```

Table 10: Consolidated comparison at $n = 2048$. Best in each row is **bold**.

Metric	Grassmann+PAT	Grassmann Dense	Transformer
Val PPL (↓)	139.44	**132.57**	517.32
Parameters	22.54 M	22.54 M	22.30 M
FFN Sparsity	**50%**	0%	0%
KV-cache needed	**No**	**No**	Yes
FP32 GPU tok/s @ 2048	42,683	48,366	23,934
INT8 CPU tok/s @ 2048	3,007	3,018	1,239
Peak VRAM @ 2048 (MB)	**219**	**219**	446
Latency @ 2048 (ms)	48.0	**42.3**	85.6

References

[1] Grassmann Flows: Stable and efficient inference for language models. *arXiv preprint* arXiv:2512.19428, 2024.
https://arxiv.org/abs/2512.19428

[2] Mishra, A., Latorre, J. A., Pool, J., Stosic, D., Stosic, D., Venber, G., and Micikevicius, P. Accelerating Sparse Deep Neural Networks. *arXiv preprint* arXiv:2104.08378, 2021.
https://arxiv.org/abs/2104.08378

[3] Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models. *ICLR*, 2017.

[4] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I. Language models are unsupervised multitask learners. *OpenAI Technical Report*, 2019.

[5] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. *NeurIPS*, 2017.